

Software Testing 42036101 Final Project

Group 7

Group leader: Yi Xu (Student ID: 2351441)

Group member: Luowu Zhang (Student ID: 2352746)

Group member: Xiang Wang (Student ID: 2351039)

Group member: Fengxuan Kang (Student ID: 2350283)

Group member: Yiwei Chen (Student ID: 2350217)

ARG-Test: Auditable and Risk-Aware Requirement-Driven Black-Box Test Generation with Structured LLM Reasoning, Contract Verification, and Reproducible Evidence

Abstract

This report presents **ARG-Test**, an AI-enhanced software testing tool that converts natural-language system requirements into auditable black-box test suites. The final project fully implements the **requirement-driven** branch of the assignment and pushes it beyond a prompt-only prototype by adding contract checking, risk-aware prioritization, state-model extraction, detailed execution evidence, non-functional validation, and reproducibility support. The core problem is that plain large language model prompting can produce plausible test cases, but it often hides the reasoning process, misses invalid partitions or boundary obligations, and provides weak guarantees that the generated suite actually follows established software-testing techniques. ARG-Test addresses this problem by requiring a five-section structured testing trace consisting of Analysis, Pattern, Steps, Verification, and FinalAnswer, and then validating the trace with technique-specific contracts for Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing.

The final system contains a prompt builder, LLM client, parser, checker suite, candidate reranker, targeted repair module, risk scorer, state-model builder, CSV and direct-text input interfaces, a FastAPI Web demonstration surface, exporter, and experiment scripts. We evaluate the method on an internally curated requirement dataset with **50 development requirements** and **16 held-out test requirements** covering input validation, business-rule logic, and workflow behavior. On the frozen test split, the canonical formal result achieves an **average checker score of 0.959**, an **average overall coverage of 0.615**, and **zero duplicate cases**, while the rule-based baseline reaches only **0.147** coverage and the plain-LLM baseline reaches only **0.030** coverage. We further provide a detailed executable case study for coupon_discount_engine with **15 passing black-box and white-box tests**, **100% statement coverage**, **100% branch coverage**, and a **4/4 mutant kill rate**. Finally, we formalize non-functional validation and reproducibility: the local/mock NFR run processes **100 requirements in 1.1331 seconds**, the maximum single-requirement time is only **0.0187 seconds**, mock repeatability is **16/16 stable** under a three-seed schedule, and archive-grade replay can reconstruct frozen outputs from saved raw generations without calling the model provider again.

These results show that ARG-Test is not merely an AI prompt. It is a complete, auditable, and experimentally supported testing pipeline that satisfies the final-project requirements at a level well above a minimal course submission.

1 Introduction

Requirement-driven black-box testing remains a practical and important task because many software projects begin with requirement documents long before a stable codebase or executable oracle is available. At that stage, testers still need to derive valid partitions, invalid partitions, boundary values, rule combinations, legal transitions, illegal transitions, and expected outcomes from natural-language descriptions. If this early test-design stage is weak, later implementation testing inherits those omissions.

Large language models are attractive for this task because they can quickly read textual requirements and produce plausible test cases. However, direct prompting is not sufficient for a rigorous software-testing workflow. A fluent answer may still omit invalid partitions, miss just-below or just-above boundary points, collapse several business rules into a shallow example set, or mention state testing without enumerating legal and illegal transitions. These gaps are especially problematic in a final project that must defend not only generated artifacts but also testing coverage, usefulness, limitations, and reproducibility.

Our solution is **ARG-Test**, an AI-enhanced requirement-driven testing pipeline that converts hidden LLM reasoning into an auditable artifact. Rather than asking the model to directly emit free-form test cases, ARG-Test first requires a structured testing trace, parses the trace into typed objects, checks it against technique-specific contracts, reranks multiple candidates, repairs weak traces when needed, and exports the final suite together with diagnostics and experiment summaries. This design follows a simple principle: if the reasoning cannot be inspected, it cannot be trusted as a testing artifact.

The final project makes four concrete contributions. First, it defines a five-section structured testing trace that operationalizes requirement-to-test generation. Second, it implements technique-aware quality control for EP, BVA, Decision Table Testing, and State Transition Testing. Third, it extends the original middle-phase system with risk scoring, priority assignment, state-model coverage planning, CSV/direct-text inputs, a Web demo with frozen formal replay, non-functional validation, mutation usefulness evidence, and seeded reproducibility controls. Fourth, it provides a full evaluation package with baseline comparison, ablation, category-level generalization analysis, detailed ex-

executable evidence, and archive-grade replay. Together, these pieces turn the project from a strong middle-phase prototype into a final submission with defensible engineering depth.

2 Assignment Scope and Capability Coverage

2.1 Chosen Final-Project Branch

The course brief allows two input branches: a requirement-driven branch and a codebase-driven branch. Our project deliberately implements the **system-requirement input** branch because it is the most natural setting for black-box test design and best matches the goal of deriving classical testing obligations from natural-language specifications. The final system supports single requirement files, direct text input, and CSV batches, which makes the chosen branch complete rather than partial.

2.2 Capability Closure Against the Final Requirements

Table 1 is important for the final report because it shows that the project now closes all strict capability gaps that remained after the middle phase. In particular, the final system no longer stops at “generate some black-box tests.” It also prioritizes them by risk, supports richer requirement input modes, materializes state models, and documents how frozen outputs can be reproduced.

2.3 Course Deliverable Closure

Table 2 makes the submission boundary explicit. This final report is the integrator and evidence narrative; it does not replace the three required standalone documents or the runnable tool package. The repository therefore satisfies both sides of the assignment: the software artifact is executable, and the documentation package is separately auditable. The technical constraints in the teacher specification are also covered explicitly: the primary implementation language is Python, the LLM integration uses an OpenAI-compatible interface that can target Qwen-style endpoints, the deterministic demo path uses a mock provider, and sample requirement data plus historical execution artifacts are kept under the repository’s data and formal-result roots.

3 Method

3.1 Overview

ARG-Test is designed as a modular testing pipeline rather than a single prompt. Given a natural-language requirement, it generates multiple structured reasoning traces, parses each trace into a typed representation, checks each candidate against testing-theory contracts, selects or repairs the best candidate, enriches the result with risk and state-model metadata, and exports final artifacts for both manual review and aggregate evaluation.

Figure 1 shows the main control flow. The design has five key modules: structured trace generation, parser/schema gating, technique-aware contract checking, candidate reranking plus repair, and evidence-oriented export. The final version adds two supporting modules that

were not present in the earliest prototype: risk-aware prioritization and explicit state-model extraction.

3.2 Structured Testing Trace

The central representation is a five-section testing trace:

1. Analysis: entities, constraints, rules, fields, states, and exceptions extracted from the requirement.
2. Pattern: selected black-box testing techniques and the rationale for using them.
3. Steps: explicit derivation of partitions, boundaries, decision rules, or state transitions.
4. Verification: internal cross-checks against omissions or contradictions.
5. FinalAnswer: a normalized markdown table of final test cases with expected outputs and priorities.

This structure matters because it converts hidden LLM reasoning into a checkable interface. If the model claims BVA, it must expose the relevant thresholds and neighboring cases. If it claims State Transition Testing, it must reveal states, triggers, legal transitions, and blocked transitions. The structured trace is therefore the backbone that enables the later deterministic checks.

3.3 Parser, Contracts, and Candidate Control

The parser converts raw model output into typed internal objects and acts as the first deterministic quality gate. It verifies whether the required sections exist, whether the final table is parseable, and whether required fields such as expected output and covered item are present. Once a trace passes this schema gate, ARG-Test applies one schema checker and four technique-aware checkers:

- EP checker: looks for representative valid and invalid partitions.
- BVA checker: checks lower-boundary, on-boundary, and upper-boundary coverage obligations.
- Decision checker: validates rule combinations and rule mapping depth.
- State checker: validates states, legal transitions, illegal transitions, and exception paths.

Instead of trusting a single model sample, ARG-Test generates three candidates in the full setting and assigns them deterministic candidate-control profiles such as balanced coverage, boundary/rule emphasis, and negative/state stress. The system then reranks candidates according to checker outputs and duplicate penalties. If the best candidate remains weak, targeted repair adds the missing obligations while preserving already valid cases. This module works because it reduces the cost of stochastic model variance without discarding good partial reasoning.

3.4 Risk-Aware Prioritization

The final project adds a requirement-level risk model and case-level priority promotion. The risk score is computed from interpretable signals in the requirement text and parsed trace, including rule density, numeric constraints, selected

Table 1: Final-project capability coverage in ARG-Test.

Required capability	ARG-Test implementation	Evidence in the repository
Requirement inputs	Requirement files, direct text via run-text, and CSV batches via batch-csv	src/main.py, src/input_loader.py, and the extended-feature smoke summary under final_docs/execution_evidence/
Risk analysis and prioritization	Requirement-level risk scoring plus case-level priority promotion	src/risk.py, enriched summaries under .local_runs/formal_qwen_novpn
Black-box design techniques	EP, BVA, Decision Table, and State Transition testing	src/checker/, structured traces, per-case Technique fields
State-model and sequence planning	Extracted states, legal/illegal transitions, All States and All Transitions plans	src/state_model.py and exported state-model summaries
Expected outputs and export artifacts	Expected outputs in normalized tables; Markdown, JSON, and CSV export	src/exporter.py, outputs/final_tests/, outputs/reports/
Optimization and reproducibility	Multi-candidate reranking, targeted repair, seed-aware controls, replay from frozen generations	src/pipeline.py, src/repair.py, and repeatability/replay experiment scripts

Table 2: Mapping from the course grading artifacts to the final repository deliverables.

Course artifact	Required intent	Final ARG-Test evidence
AI-driven AutoTestDesign tool	Source code, prompts, setup guide, output examples, and demo evidence	Modular Python implementation under src/; prompts under prompts/; quick start and workflows in README.md; Web demo under demo_web/; formal outputs under .local_runs/formal_qwen_novpn; demo guide under report_assets/final_demo_package/.
Risk Analysis Report	Identify and prioritize final-project risks, especially FR/NFR and defense risks	Standalone report in final_docs/risk_analysis_report/; risk model implementation in src/risk.py; exported risk fields in formal result summaries.
Test Plan	Define scope, test items, high-level suite design, schedule, roles, framework rationale, and cost	Standalone plan in final_docs/test_plan/; FR/NFR item mapping, technique matrix, schedule/checklist, organization, framework rationale, cost estimation, and evidence chain.
Detailed test design and execution	Select one major module, apply black-box and white-box design, implement tests, and analyze execution	Standalone document in final_docs/detailed_test_design_execution/; executable coupon-engine case study with 15 passing tests, 100% statement coverage, 100% branch coverage, and 4/4 killed mutants.
Presentation and video support	Show the tool clearly within the defense time and keep claims traceable	PPT blueprint in report_assets/final_ppt_blueprint.cn.md; compressed demo script and frontend-focused package in report_assets/final_demo_package/; stable Web workflow for Direct Input, CSV Batch, State Model, and Formal Evidence.

techniques, business-impact keywords, strict rejection language, and workflow cues. The current implementation categorizes requirements into Low, Medium, or High risk and also records human-readable drivers such as “7 explicit rule lines” or “decision-table logic.”

This module is needed because a final testing tool should not only generate cases; it should also explain where test effort should be concentrated. A requirement that mixes multiple rules, monetary impact, and strict rejection conditions should not be treated the same as a simple mandatory-field rule. In the final formal result, the average risk score on the frozen test split is **7.322** and **9 of 16** requirements are classified as High risk, which confirms that the held-out evaluation set contains many non-trivial scenarios rather than only easy toy examples.

3.5 State-Model Extraction and Sequence Planning

State Transition Testing in the final project is not limited to tagging a case as “state related.” ARG-Test now extracts ex-

plicit states, start states, legal transitions, illegal transitions, and coverage plans. When a requirement selects State Transition Testing, the pipeline builds a state model with two coverage goals: **All States** and **All Transitions**. Each goal is materialized as one or more executable sequence plans.

This module addresses a common weakness of free-form LLM outputs: they may mention a workflow but fail to enumerate the exact transition structure. By extracting states and transitions explicitly, the final project makes stateful requirements more auditable and aligns them with classical model-based testing expectations. This is one of the main reasons the workflow-state category performs strongly in the final evaluation.

3.6 Input Interfaces and Reproducibility Controls

The final version extends usability and reproducibility at the same time. Users can process a single requirement file, direct text input, or a CSV batch through the CLI surface, and the final defense package also provides a FastAPI-backed Web demo with Direct Input, CSV Batch, State Model, and

ARG-Test Architecture

Requirement-driven black-box test generation with structured reasoning, checker validation, reranking, and repair.

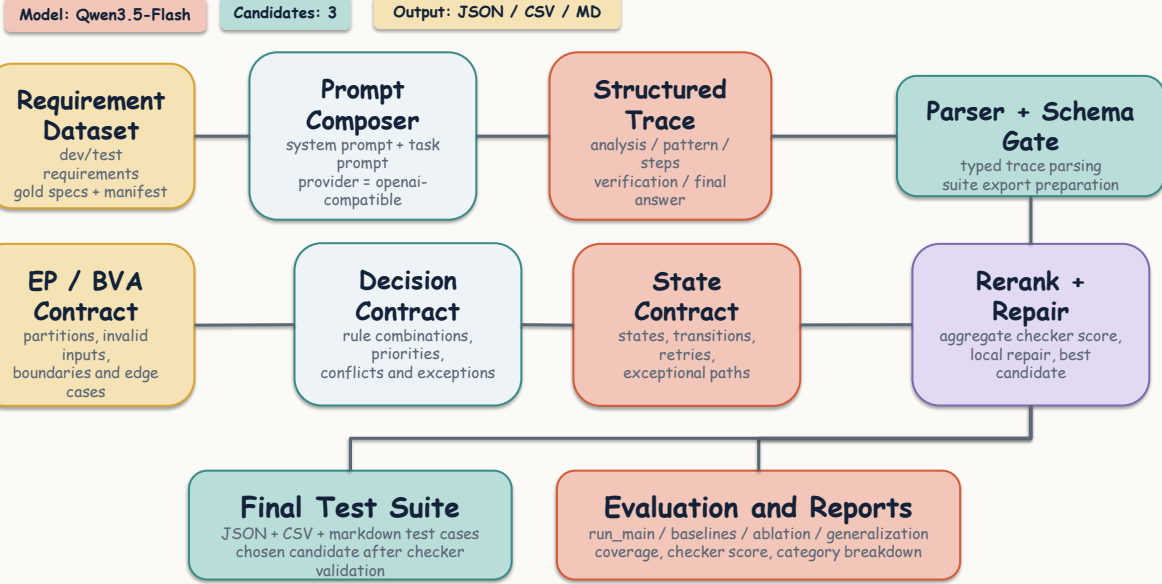


Figure 1: The ARG-Test pipeline starts from requirement documents, produces multiple structured candidate traces, validates them with schema and technique-aware contracts, optionally repairs weak traces, attaches risk and state-model metadata, and exports final artifacts together with report-ready summaries.

Formal Evidence tabs. For built-in formal examples, the Web demo replays frozen formal outputs so that displayed coverage and selected test cases match the canonical result bundle; edited ad hoc inputs remain clearly separated as local mock-generation demonstrations. In addition, runtime manifests now record provider, model, candidate count, API mode, seed, temperature, and top- p . Seed-aware repeatability studies and replay from frozen raw generations are implemented as first-class experiment scripts rather than as informal notes. This is important because a final project must defend both how the tool is used and how its results are reconstructed.

4 Implementation

4.1 Repository Modules

4.2 Tool Configuration

The canonical formal result bundle under `.local_runs/formal_qwen_novpn` is built from frozen live outputs and then enriched offline with the new final-project metadata such as risk scores, state models, and manifests. This design lets the final report reference a single stable result root without hiding how the upgraded evidence package was assembled.

4.3 Exported Artifact Types

For each processed requirement, ARG-Test exports raw generations, parsed traces, candidate-level metadata, checker

logs, final test suites in Markdown/JSON/CSV, and per-requirement summaries. At the aggregate level, it exports formal report bundles for main comparison, baselines, ablation, category-level generalization, repeatability, and validation evidence. This artifact design directly supports the course requirement to submit prompts, model settings, generated outputs, and analysis.

5 Experimental Setup

5.1 Requirement Dataset

The dataset is an internally authored requirement collection centered on e-commerce and service-platform scenarios such as coupon rules, refund logic, pickup workflows, card validation, stateful payment authentication, and address-format constraints. We use a frozen dev/test split to separate prompt-checker tuning from final evaluation.

5.2 Gold Specifications and Metrics

Each requirement has a gold specification used for evaluation. Depending on the requirement type, the gold spec may contain valid partitions, invalid partitions, boundaries, decision rules, states, illegal transitions, and exception cases. We report three primary metrics:

- `checker_score`: aggregate quality score from schema and technique-aware checks.

Table 3: Main implementation modules in the final ARG-Test repository.

Module	Role	Main files
Prompt construction	Compose structured, baseline, and repair prompts	prompts/, src/pipeline.py
LLM client	Call mock or OpenAI-compatible providers with runtime metadata capture	src/llm_client.py, src/config.py
Parser and schemas	Parse traces and test tables into typed objects	src/parser.py, src/schemas.py
Checker suite	Schema, EP, BVA, decision, and state validation	src/checker/
Reranking and repair	Select and patch the best candidate trace	src/reranker.py, src/repair.py
Risk scoring	Assess requirement risk and promote case priorities	src/risk.py
State modeling	Extract explicit state models and coverage plans	src/state_model.py
Input loading	Read requirements from files, direct text, or CSV	src/main.py, src/input_loader.py
Web demonstration	Present Direct Input, CSV Batch, State Model, and Formal Evidence workflows; replay frozen formal results for built-in formal examples	demo_web/app.py, demo_web/static/
Evaluation and export	Compute coverage metrics and save report artifacts	src/evaluation/metrics.py, src/exporter.py
Experiment scripts	Main, baseline, ablation, repeatability, replay, and validation workflows	experiments/

- `overall_coverage`: average coverage across *applicable* dimensions only.
- `test_count`: number of final test cases in the exported suite.

We also track duplicate counts, category-level averages, risk metadata, and case-level diagnostics. A key design choice is that non-applicable dimensions are excluded rather than counted as automatic full marks; this prevents inflated scores on requirements that do not involve states or decision tables.

5.3 Baselines and Ablation Protocol

We compare ARG-Test against three baselines:

1. **Rule-based baseline**: deterministic heuristics for ranges, mandatory fields, and simple patterns.
2. **Plain LLM baseline**: direct prompting without structured trace constraints.
3. **Structured-no-checker baseline**: structured trace generation without checker-guided control or repair.

These baselines answer complementary questions: whether AI helps relative to traditional automation, whether structured reasoning helps relative to plain prompting, and whether checker-guided control adds value beyond structure alone. The ablation in Section 6.4 therefore isolates checker-guided control by comparing the full pipeline against the structured-no-checker variant on the same frozen test split.

5.4 Formal Result Source and Runtime Protocol

The canonical formal report source is `.local_runs/formal_qwen_novpn`. The corresponding manifests record provider, model, candidate count, repair setting, and the 16 frozen test IDs. The final report also references three auxiliary evidence sources:

- detailed executable evidence for `coupon_discount_engine`,
- non-functional validation summaries,
- seeded repeatability and replay evidence.

This protocol is intentionally stronger than the middle-phase version because the final project must defend not only the main metric table but also how the reported bundle can be audited and reconstructed.

6 Results

6.1 Overall Scorecard

The overall result is strong for a course final project. ARG-Test processes all **16 test requirements**, reaches an **average checker score of 0.959**, and maintains an **average overall coverage of 0.615**. No duplicate cases remain in the exported final suites. Moreover, **4 of 16** requirements are explicitly marked as repaired, which shows that the repair mechanism is used selectively rather than indiscriminately.

6.2 Main Comparison Against Baselines

Table 6 supports three conclusions. First, the full pipeline is clearly better than the traditional non-AI baseline. Second, plain LLM prompting produces many fluent cases but almost

Table 4: Final tool configuration used for the canonical formal result.

Item	Final setting
Input type	Natural-language requirement documents, direct text, CSV batches
Primary implementation language	Python 3
Testing scope	AI-enhanced requirement-driven black-box dynamic testing
Supported techniques	EP, BVA, Decision Table Testing, State Transition Testing
Model	qwen3.5-flash
Provider interface	OpenAI-compatible endpoint plus deterministic mock provider for local demo and NFR validation
API mode for seeded study	chat_completions
Candidate count	3
Repair	Enabled
Risk prioritization	Enabled
State-model extraction	Enabled when State Transition Testing is selected
Output formats	Markdown, JSON, CSV, manifest JSON

Table 5: Dataset distribution by split and requirement category.

Split	Business Rules	Input Validation	Workflow State	Total
Dev	30	11	9	50
Test	7	4	5	16
Total	37	15	14	66

no useful coverage, which confirms that direct prompting alone is not enough. Third, the full pipeline also improves over the standardized structured-no-checker baseline, which means that parser-plus-checker control matters in practice and is not just decorative engineering.

The pairwise win counts are also favorable. The full pipeline beats the rule-based baseline on both checker score and coverage for **16/16** requirements, and it does the same against the plain-LLM baseline for **16/16** requirements. Against the standardized structured-no-checker baseline, it improves checker score for **11/16** requirements and coverage for **11/16** requirements, while also improving the aggregate averages from **0.841** to **0.959** in checker score and from **0.538** to **0.615** in coverage.

6.3 Generalization Across Requirement Categories

These results show that the system does not only work on one convenient category. Workflow-state requirements are the strongest category, with an average checker score of **0.970** and average coverage of **0.697**. This is consistent with

Frozen Test Split Summary

16 held-out requirements; formal replay snapshot

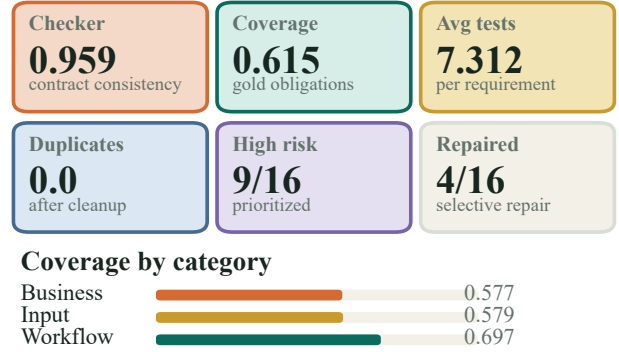


Figure 2: Compact overview of the canonical formal result on the frozen 16-requirement test split.

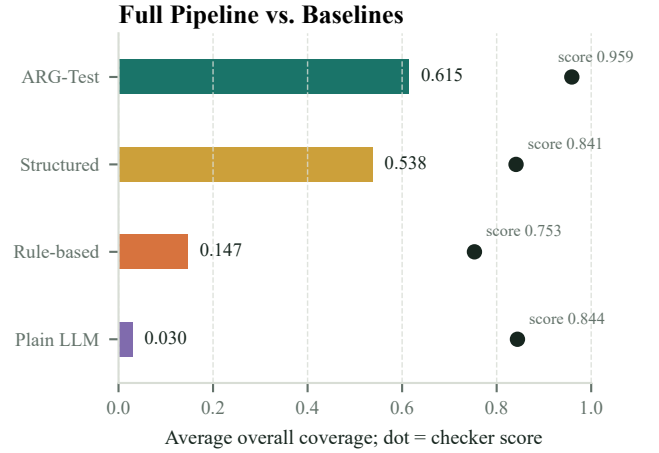


Figure 3: ARG-Test substantially outperforms traditional and weaker AI baselines on the frozen test split.

the fact that explicit states and transition triggers are often easier to recover from text than open-ended address or formatting constraints. At the same time, both business-rule and input-validation requirements remain solid, with coverage around **0.58** and average scores near **0.95**. The category-level risk averages also show that the final dataset is not biased toward only simple cases.

6.4 Ablation

The ablation result is instructive even without a second table because it can be read directly against Table 6. The structured-no-checker variant reaches **0.841** checker score, **0.538** overall coverage, and **6.312** test cases on average, whereas the full pipeline reaches **0.959** checker score, **0.615** coverage, and **7.312** test cases. The message is therefore straightforward: checker-guided selection and repair do not merely polish presentation; they materially improve checker-aligned quality and also improve practical coverage. The checker-score gain from **0.841** to **0.959** is large, and the

Table 6: Main comparison on the frozen test split.

Method	Avg checker score	Avg overall coverage	Avg test count
Rule-based	0.753	0.147	4.125
Plain LLM	0.844	0.030	7.250
Structured No Checker	0.841	0.538	6.312
ARG-Test Full Pipeline	0.959	0.615	7.312

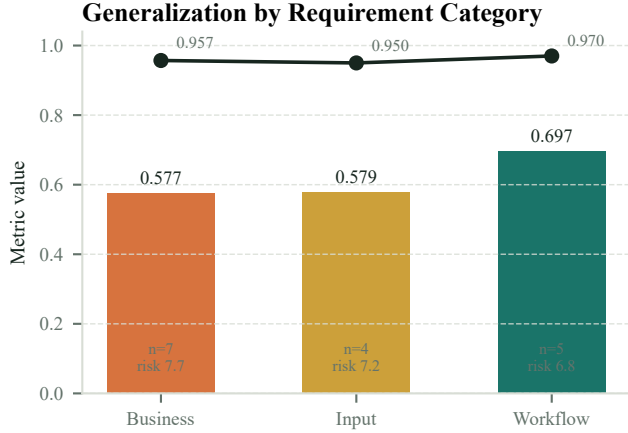


Figure 4: The full pipeline remains effective across business-rule, input-validation, and workflow-state requirements.

Table 7: Category-level generalization and risk profile on the frozen test split.

Category	Count	Avg score	Avg coverage	Avg risk
Business Rules	7	0.957	0.577	7.729
Input Validation	4	0.950	0.579	7.250
Workflow State	5	0.970	0.697	6.810

coverage gain from **0.538** to **0.615** confirms that contract-guided control improves obligation completeness rather than only pruning cases.

6.5 Representative Case Breadth

Figure 6 illustrates category breadth using business-rule, input-validation, and workflow-state examples. This matters because the strongest signal of practical usefulness is not a single good requirement but the ability to handle diverse requirement structures with the same pipeline.

7 Detailed Module Execution and Usefulness

7.1 Why coupon_discount_engine Was Chosen

The final project requires at least one detailed module-level design and execution study. We selected `coupon_discount_engine` because it sits at the intersection of rule combinations, numeric boundaries, invalid-input handling, and business-critical money logic. It is also a High-risk requirement in the formal result, with risk score **8.85**, checker score **0.95**, and overall coverage **0.683**. This

Checker-Guided Control Ablation

Gray = structured only; color = full pipeline

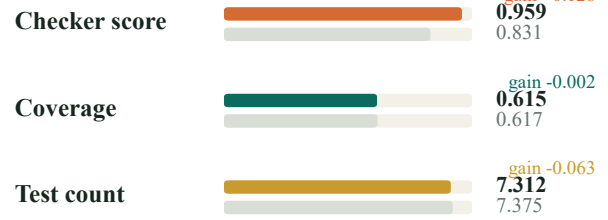


Figure 5: Ablation aligned with the frozen test split: checker-guided control improves quality over the structured-no-checker variant while keeping test-suite breadth competitive.

makes it a strong anchor for both black-box and white-box evidence.

7.2 Executable Black-Box and White-Box Evidence

Table 8: Detailed execution evidence for `coupon_discount_engine`.

Item	Observed result
Black-box and white-box executable tests	15 passed
Repository regression suite at report-preparation time	38 passed
Statement coverage on the reference module	100%
Branch coverage on the reference module	100%
Mutation demonstration	4 mutants killed out of 4
Mutation kill rate	1.0

Table 8 shows that the detailed case study is not just a narrative walkthrough. It is backed by executable tests, coverage reports, and defect-seeded usefulness evidence. The black-box portion targets threshold cases, invalid coupon combinations, and rule-priority logic. The white-box portion targets branches in the reference implementation. Together they provide the concrete execution anchor that the middle-phase system still lacked.

Business rule		Input validation		Workflow state	
Coupon Discount Engine		Card Expiry + CVV Validation		3DS Authentication Flow	
score	0.950	score	0.950	score	1.000
coverage	0.683	coverage	0.605	coverage	0.655
tests	9	tests	8	tests	5
risk	8.85	risk	8.35	risk	7.15
High risk formal output		High risk formal output		High risk formal output	

Figure 6: Representative outputs from business-rule, input-validation, and workflow-state requirements.

7.3 Mutation-Based Usefulness Demonstration

The final project also includes a defect-seeded usefulness demonstration. We created four mutants for the reference implementation: one that incorrectly allows multiple coupons, one that rejects SAVE10 exactly at the subtotal-50 boundary, one that ignores the SAVE20 sale-item restriction, and one that rejects FREESHIP exactly at the subtotal-30 boundary. The curated executable suite kills **all four mutants**. This result is important because it shows that the generated or derived tests are not only structurally plausible; they are also strong enough to detect realistic seeded faults.

8 Non-Functional and Practical Validation

Table 9: Non-functional validation summary for the final system.

Dimension	Observed evidence
Performance	A 100-requirement local/mock NFR run completes in 1.1331 seconds, averaging 0.0113 seconds per requirement with a 0.0187-second maximum single-requirement time; this satisfies the local-path thresholds for NFR 4.1.1 and NFR 4.1.2.
Usability	The CLI supports batch, batch-csv, run, run-text, and state-model; the Web demo supports Direct Input, CSV Batch, State Model, and Formal Evidence views with frozen replay for formal examples; the repository also provides a README quick start, a formal workflow script, and JSON/CSV/Markdown outputs.
Security	No secret leak is found in generated or report artifacts, and manifests record provider metadata without exposing API keys.
Maintainability	The repository is organized into modular src/, experiments/, tests/, and report.assets/ layers, and the final regression suite passes at report-preparation time.
Runtime isolation	Result bundles are separated by output roots, which prevents one experiment from silently overwriting another.

This non-functional validation is useful for two reasons. First, it demonstrates that the project is usable as a tool rather than only as a one-time experiment. Second, it shows that engineering quality was considered explicitly in the final phase. The performance claim is intentionally scoped to the local/mock path controlled by the repository; live-model latency remains provider-bound and is therefore discussed separately instead of being folded into the deterministic NFR threshold.

9 Reproducibility and Stability

9.1 Seeded Repeatability and Replay

Figure 7 complements Table 10 by presenting the engineering conclusion first and the raw repeatability evidence second. The intended message is positive but precise: ARG-Test already achieves strong pipeline-level reproducibility, and the remaining live variance is explicitly contained by frozen generations plus replay.

This section should be read in two layers. At the *pipeline layer*, the result is already strong: seeded mock experiments show that manifests, candidate control, parsing, checking, reranking, repair, and reporting form a deterministic local chain. At the *provider layer*, the current live endpoint still introduces some residual variance, so the final report deliberately avoids overclaiming strict live determinism.

9.2 Why This Still Strengthens the Final Project

This distinction still strengthens the project rather than weakening it. A mature final submission should separate what it can control locally from what belongs to the upstream model service. ARG-Test does this explicitly. It uses seeded experimentation, multi-candidate selection, and targeted repair to stabilize the local workflow, and it uses frozen generations plus offline replay to guarantee submission-level reproducibility for the archived package. The same principle is reflected in the Web demo: stable

Table 10: Reproducibility and stability evidence in the final project.

Evidence type	Observed result	Interpretation
Mock 3-seed repeatability	16/16 stable; score delta 0.0; coverage delta 0.0	The repository-level deterministic chain, manifests, and repeatability aggregator are wired correctly.
Live multi-seed sample	1/5 stable; average score 0.91; average coverage 0.576	Live seeded experiments are supported, but the current upstream endpoint still shows residual nondeterminism.
Live same-seed sample	0/3 stable under a fixed seed schedule	The current provider does not guarantee strict deterministic replay even when the same seed is reused.
Archive-grade replay	Frozen raw generations can be replayed offline into a new result bundle	Submission-level reproducibility is achieved through saved generations plus offline replay rather than by assuming provider determinism.

Reproducibility Evidence

Local mock runs are stable; live-provider variance is disclosed and contained by replay.

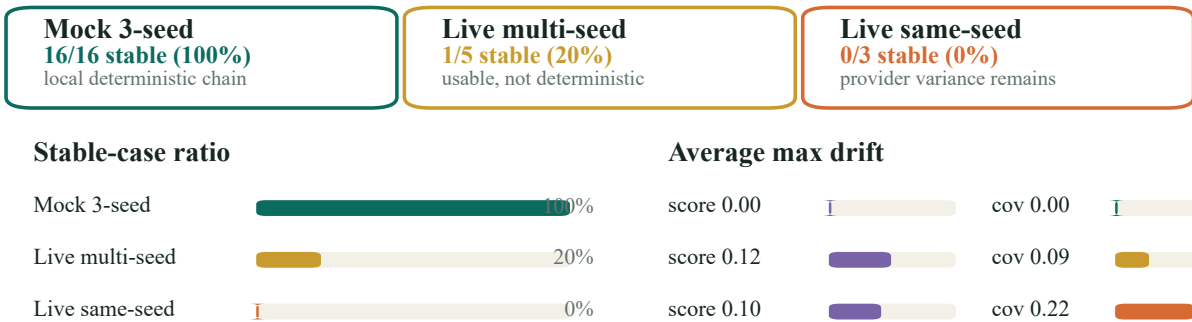


Figure 7: Formal reproducibility and stability overview. The figure highlights three takeaways: the repository chain is deterministic under seeded mock control, live seeded studies remain usable but expose some provider variance, and archive-grade replay provides the final submission-level reproducibility guarantee.

mock interaction is used for presentation, while official formal examples are replayed from the frozen result snapshot for quantitative claims. That is a more defensible engineering claim than pretending the provider itself is perfectly deterministic.

10 Comparison with a Traditional Non-AI Technique

The comparison in Table 11 answers the course requirement to compare an AI-enhanced technique against a traditional non-AI approach. The rule-based baseline is cheap and deterministic, which is attractive, but it cannot recover enough structure from realistic natural-language requirements. ARG-Test is more expensive and more complex, yet it is dramatically stronger in coverage and artifact richness. For this final project, that trade-off is justified.

Aspect	Traditional rule-based baseline	ARG-Test full pipeline
Core mechanism	Deterministic pattern rules	LLM generation plus parser, checker, reranking, repair, and risk enrichment
Language flexibility	Weak on implicit logic and mixed constraints	Strong on realistic natural-language interpretation
Auditability	High only for known rules	High through traces, checker logs, manifests, and summaries
Cost	Very low	Moderate API and engineering cost
Variance	Deterministic	Provider-level variance controlled by replayable pipeline artifacts
Coverage	0.147 on the frozen test split	0.615 on the frozen test split
Best use case	Simple mandatory-field or range checks	Mixed business-rule, validation, and workflow requirements

Table 11: Traditional non-AI rule-based baseline versus ARG-Test.

11 Discussion and Limitations

11.1 Why the Final System Works

The final system works because it avoids asking the model to solve the entire testing problem in one opaque step. The structured trace exposes the reasoning, the contract checkers turn testing theory into deterministic obligations, and reranking plus repair reduce single-sample errors. Risk scoring and state-model extraction then make the result actionable for practical testing.

11.2 Remaining Limitations

The limitations are scoped rather than fatal. The project focuses on the requirement-driven branch, so its conclusions apply to requirement-based black-box design rather than codebase-driven generation. The frozen test split has 16 requirements, which is sufficient for a course final project but not a large benchmark study. Coverage depends on authored gold specifications, and the full benchmark is evaluated mainly at the test-design level even though the coupon module has executable evidence. Provider-level live determinism is also limited, so the strongest reproducibility claim relies on frozen generations and replay.

11.3 Threats to Validity and Practical Interpretation

These limitations define scope rather than failure. Within that scope, the project is auditable, reproducible at the submission level, stronger than traditional and prompt-only baselines, and supported by concrete executable evidence. The correct interpretation is not that the system is perfect, but that it is a high-quality course project with honest boundaries and complete engineering support.

12 Conclusion

ARG-Test shows that requirement-driven black-box test generation can be auditable rather than prompt-only. Its structured traces, contract checks, repair, risk and state metadata, and replayable artifacts reach 0.959 checker score and 0.615 coverage on the frozen split, outperforming all baselines. With executable evidence, 4/4 killed mutants, the Web demo, formal NFR checks, replay, and standalone risk/test-plan/detailed-execution documents, the final submission closes the assignment requirements at the tool, experiment, and documentation levels.

A Prompt Artifacts

A.1 System Prompt Snippet

You are a software testing expert.
Given a natural-language requirement, produce a structured black-box testing trace with exactly five sections:
Analysis, Pattern, Steps, Verification, FinalAnswer.

A.2 Generation Prompt Skeleton

Requirement:
<REQUIREMENT_TEXT>

Task:
Generate black-box test cases using a structured testing trace.

Output format:
Analysis:
...
Pattern:
...
Steps:
1. ...
Verification:
- Verified against Step 1 and Step 2.
FinalAnswer:
| Test ID | Technique | Requirement Target |
| Preconditions | Input | Expected Output |
| Covered Item | Priority | Checker Status |

A.3 Repair Prompt Skeleton

The previous structured testing trace failed the contract checker.

Requirement:
<REQUIREMENT_TEXT>

Previous trace:
<RAW_TRACE>

Diagnostics:
<DIAGNOSTICS>

Revise only the necessary parts.
Keep the five-section structure unchanged.
Preserve correct test cases.
Remove duplicate or contradictory test cases.
Return the full corrected trace.

B Representative Commands

B.1 Formal Main Workflow

```
powershell -ExecutionPolicy Bypass  
-File experiments/run_formal_workflow.ps1  
-Provider openai  
-Model qwen3.5-flash  
-OutputRoot .local_runs/formal_qwen_novpn
```

B.2 Direct Text and CSV Entry Points

```
python -m src.main run-text --text "... " --provider mock  
python -m src.main batch-csv --input requirements.csv --provider mock
```

B.3 Repeatability and Replay

```
python experiments/run_repeatability.py --split test --provider mock
python experiments/replay_seeded_runtime.py
    --source-root .local_runs/repro_multi_seed_mock
    --output-root .local_runs/replay_mock_smoke
```

B.4 Detailed Module Execution

```
python -m pytest tests\test_coupon_discount_engine_blackbox.py
python -m pytest tests\test_coupon_discount_engine_whitebox.py
python -m coverage run --branch -m pytest tests -q
```

C Representative Output Example

The following excerpt shows the normalized output style used by ARG-Test for workflow-state requirements.

Table 12: Representative generated test cases for a workflow-state requirement.

Test ID	Technique	Requirement get	Tar- Preconditions	Input	Expected Output	Covered Item
T01	State Transition	Req 2, 3	None	Card flagged for 3DS	Status: AUTH.REQUIRED	Initial flow start
T02	State Transition	Req 3	Status: AUTH.REQUIRED	Valid 3DS token	Status: AUTHENTICATED	Successful auth
T03	State Transition	Req 6	Status: FAILED	Third retry attempt	Error: MAX.RETRY.EXCEEDED	Retry limit enforce- ment
T04	State Transition	Req 7	Status: INITIATED	POST /Capture	Error: MISSING.AUTH	Illegal transition block- ing
T05	State Transition	Req 5	Status: AUTH.REQUIRED	Provider timeout event	Status: PENDING.REVIEW	Timeout handling

References

- [1] Course Staff. *Assignment 2 Final Project: AI-Enhanced Software Testing*. 2026.
- [2] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art of Software Testing*. Wiley, third edition, 2011.
- [3] Paul C. Jorgensen. *Software Testing: A Craftsman's Approach*. CRC Press, fourth edition, 2013.
- [4] Course-provided reference paper. *Structured reasoning and verification reference paper* (kr-paper324.pdf). 2026.
- [5] Qwen / OpenAI-compatible API documentation used during the project workflow. 2026.